BECKETT

DESIGN PROPOSAL · OPS-151 · --project beckett

Run of Show
a workflow engine the concierge conducts

A proposal to retire Plane as Beckett's execution backend and replace its one fixed ticket lifecycle with a per-task, concierge-authored workflow: stages, iteration budgets, parallel fan-out and gates composed at filing time — not compiled into the dispatcher.

Author
Beckett — worker seat
Claude Fable 5 · high

Requested by
ro (owner)
Discord · OPS-151

Date
12 July 2026
rev 1

Status
Proposal — no code
grounded in v3.2 source

IN ONE PAGE

TL;DR

Every piece of work Beckett takes on today — a copy tweak, a concurrency fix, a self-modification — marches through the same compiled-in pipeline: `todo → in_progress → in_review → done`, one implement seat, at most one review seat, three rework cycles, one live worker per ticket. That shape lives half in Plane (a remote SaaS ticket board we poll every few seconds) and half in a 3,200-line dispatcher switch statement. Changing it means changing Beckett's source and redeploying: adding *one* design stage cost us a new board, two new enum values, and surgery in five modules (OPS-111).

This document proposes making the workflow itself **data**: a small per-task DAG — the *run of show* — that the concierge writes when it files work. Stages, iteration caps, parallel fan-out, join rules and human gates become fields in a spec, interpreted by a generic engine that keeps everything already proven: worktrees and the task/branch tree, per-stage casting, review gates, the journal, the courier, steering. Plane is first demoted to a read-only mirror, then retired.

HOW TO READ THIS

Everything in §1 is fact, verified against the working tree at commit `6fac13d` with `file:line` receipts. Everything from §3 on is proposal. The two legacy lifecycles reappear in Appendix A as flow specs, which is the backward-compatibility argument in one page: if the engine can run *those two specs* identically, cutover changes nothing until we ask it to.

WHAT EXISTS · VERIFIED AGAINST SOURCE

1

Ground truth — how Plane runs Beckett today

Beckett's execution backend is a self-hosted **Plane** instance (`plane.0xbeckett.me`). The concierge files work as Plane issues; a poller diffs the board into events; a dispatcher turns events into worker processes in git worktrees. One paragraph of architecture, five subsystems of detail.

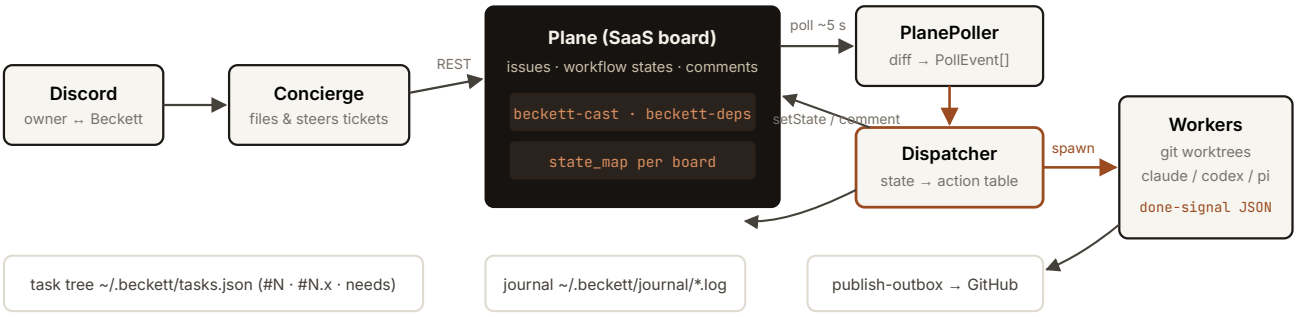

Fig. 1 — the v3 loop. The concierge writes to Plane; the poller diffs Plane into events; the dispatcher spawns workers and writes state back. All durable local artifacts (task tree, journal, outboxes) already live outside Plane.

1.1 The state machine and its enum

`TicketState` is a fixed eight-value union `src/plane/types.ts:26` — `backlog` , `todo` , `design` , `design_review` , `in_progress` , `in_review` , `done` , `cancelled` — with `done` / `cancelled` terminal `types.ts:39`. Each Plane board carries a `state_map` renaming these onto its columns `src/config.ts:129`. The dispatcher consumes poll events through one hardcoded dispatch table `src/dispatch/dispatcher.ts:1003-1062`:

EVENT / STATE ENTERED	GUARD	ACTION
`design`	INT board only, no live worker	spawn stage `design`
`in_progress`	no live worker	spawn stage `implement`
`in_review`	no publish hold, no live worker	spawn stage `review`
`done`	—	reap · drain orphaned steers · `promoteDependents()`
`todo` / `backlog` / `design_review`	—	park: abort+reap, commit WIP, keep worktree
`cancelled`	—	abort · reap · dispose
`comment_added`	live worker, author ≠ Beckett	`handle.nudge(body)` — steering

Stage names are string literals switched on in three places: the spawn table above, `castFor()` `dispatcher.ts:1998-2005` and the worker-done router `dispatcher.ts:2152-2161` (`design` , `design_check` , `implement` , `review`). A new stage is therefore a source change in at least three switch statements plus the enum, the poller's recovery list, and every board's `state_map` .

1.2 Casting, effort and the review gate

Casting is per-stage: a ticket description carries a fenced `beckett-cast` JSON block mapping stage name → `HarnessSpec` (`{harness, model, effort, reviewTier}`) `src/plane/cast.ts:29-59`. The `Casting` record is deliberately open-ended — arbitrary stage names already type-check `src/plane/types.ts:71-75` — but only the four hardcoded stage strings are ever spawned. Two consequential rules:

- **The review gate is inferred from a model knob.** `reviewTierFor()` `dispatcher.ts:2475-2479`: implement effort `low|medium` → `self` (one pass, straight to done); `high|xhigh|unset` → `fresh` (spawn an adversarial reviewer). An explicit `reviewTier` field overrides — a patch bolted on precisely because effort was overloaded.
- **Effort also sets the envelope** — turn caps and wall-clock via `ENVELOPE_BY_EFFORT` `src/dispatch/spawn.ts:217-222` (low 15 turns/10 min … xhigh 100 turns/60 min).

Named cast presets live in `~/.beckett/presets.json`, hot-reloaded on every filing `src/plane/presets.ts:76-112` — a precedent this proposal leans on: *per-task behaviour as user-editable data worked*.

1.3 Iteration policy: four global constants

LOOP	CAP	ON EXHAUSTION
review fail → re-implement (“rework”)	`MAX_REWORK_CYCLES = 3` `dispatcher.ts:199`	comment, park in `in_review` for a human
implement crash/timeout retry	`MAX_IMPLEMENT_RETRIES = 3` `dispatcher.ts:209`	publish WIP, move to `todo`
reviewer infra failure	`MAX_REVIEW_INFRA_RETRIES = 1` `dispatcher.ts:212`	park in `in_review`
design ↔ completeness check	`MAX_DESIGN_CYCLES = 2` `dispatcher.ts:202`	escalate to `design_review` with ⚠ note

These are compile-time constants. No ticket can ask for one shot, or six; the concierge’s only lever over iteration is not filing the work.

1.4 One worker per ticket, worktrees, done-signals

The dispatcher enforces **at most one live worker per ticket** (the `workers` map, staffing dedup and FIFO pending queue `dispatcher.ts:450, 1313-1379`) under a global `max_workers` cap. A spawn allocates a persistent worktree at `<repo>/.beckett/worktrees/<id>` on branch `beckett/task-N[.x]`, installs the scope-guard and the done-signal schema, captures the review base SHA on first implement `dispatcher.ts:1573-1586`, and periodically checkpoint-commits live work (OPS-125) `dispatcher.ts:640-709`. Workers finish by emitting a structured done-signal `{status: complete|blocked|partial, summary, filesChanged, checksRun, blockedReason}` `spawn.ts:198-209` — the single edge the state machine branches on.

1.5 Steering, journal, courier, task tree

- **Steering.** A human comment on an in-flight ticket becomes a live `nudge` (receipts: delivered / queued / will-restart / dropped); with no live worker it is buffered in `pendingSteers`, persisted, and folded into the next worker's brief `dispatcher.ts:1070–1158, 1609–1635`. Plane comments are the transport; Discord is where the human actually typed.
- **Journal.** An append-only per-ticket log under `~/.beckett/journal/<ticket>.log`, fed fire-and-forget from worker events, read on demand via `beckett journal <ticket> --tail N` `src/progress/journal.ts`. Already fully local; already richer than Plane's comment trail.
- **Courier / publish.** Finishing a ticket publishes to GitHub via a durable JSONL outbox with 1 m/5 m/30 m retries; `dispatcher.courier(id)` hands exclusive publish ownership to a human `src/dispatch/publish-outbox.ts, dispatcher.ts:831–839`. Plane state writes themselves go through a second durable outbox (`advance-outbox`) because remote writes fail often enough to need one `src/dispatch/advance-outbox.ts`.
- **Task tree.** The user-facing `#N` / `#N.x` tree with `needs` dependencies lives in `~/.beckett/tasks.json` — a local, locked, versioned registry that already mirrors every Plane state into a branch status (`backlog→waiting, … in_review→review`) `src/task/store.ts:141–152`. Cross-ticket DAG promotion (`blockedBy` → promote on done) is dispatcher logic, not Plane's `dispatcher.ts:2889–2926`.

THE LOAD-BEARING OBSERVATION

Plane contributes exactly four things: issue storage, a state column per ticket, comments-as-transport, and a human-viewable board. Everything Beckett actually *runs on* — worktrees, casting, envelopes, done-signals, steering buffers, journal, outboxes, the #N tree, DAG promotion — is already local, already durable, and already ours. The migration in §6 is small precisely because of this.